

Participation to the Perceval Quest

Valentin DEUMIER

Mines de Paris - PSL, France

March 2025

1 Overview

The Perceval Quest is a competition hosted by Quandela, a french start-up specialized in photonic quantum computing, along with Scaleway, a cloud provider for large computing. The goal of the competition is to show the potential of Linear Optical Quantum Computing (LOQC) to tackle classification problems.

The dataset considered is the famous MNIST hand-written digits dataset. In order to demonstrate the ability of quantum computing to train on small datasets, only 6000 images are used in the training dataset.

The algorithm will be coded in Python, and the optical circuits will be simulated using the `perceval` library provided by Quandela.

In the results section, the performances of my LOQC model will be compared against a standard Convolutional Neural Network.

Training data:			Validation data:		
Class	Count	Percentage	Class	Count	Percentage
0	604	10.07%	0	49	8.17%
1	725	12.08%	1	74	12.33%
2	619	10.32%	2	67	11.17%
3	631	10.52%	3	49	8.17%
4	559	9.32%	4	64	10.67%
5	535	8.92%	5	66	11.00%
6	572	9.53%	6	55	9.17%
7	622	10.37%	7	54	9.00%
8	564	9.40%	8	52	8.67%
9	569	9.48%	9	70	11.67%

Example digits from the training data:

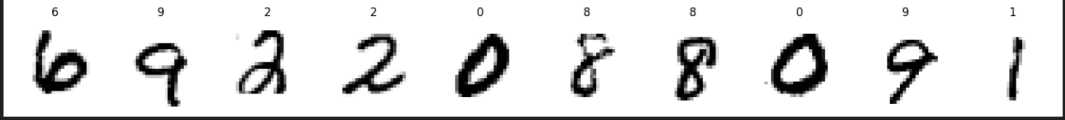


Figure 1: Description of the used dataset

2 Ideas and method

2.1 Encoding classical data into an optical system

The first step in order to construct a LOQC algorithm is to encode the classical data (in this case, 28 by 28 greyscale images) into an optical system. An optical system can be thought of photons traveling along spatial modes, and interacting between them and with optical components.

I have decided not to encode the data into the photons, as the data is too highly dimensional, and it would require too much photons for our computing ability. I have therefore decided to encode the data into the optical components.

A very useful property of linear optical systems is that any unitary matrix can be decomposed as an optical circuit. This allows for data encoding without losing its bidimensional aspect (in other words, there is no need to flatten the image).

I tried two methods to construct a unitary matrix from a given greyscale image:

- As inspired by the solution to a time-independent Schrödinger's equation, the transformation :

$$\mathbf{X} \rightarrow \exp(it \frac{\mathbf{X} + \mathbf{X}^T}{2}), \quad \mathbf{X} \in \mathcal{M}_n(\mathbb{R})$$

outputs a unitary matrix of size n , with t a real tunable parameter. However, half of the information is lost during the process, as only the symmetric part of the matrix is used.

- The *unitary dilation theorem* [1], as follows:

If $A \in \mathcal{M}_n(\mathbb{C})$ verifies $\|A\| \leq 1$, then $U_A = \begin{pmatrix} A & \sqrt{I - AA^\dagger} \\ \sqrt{I - A^\dagger A} & -A^\dagger \end{pmatrix}$ is a $2n$ by $2n$ unitary matrix.

allows for the construction of a unitary matrix from normalized data. This is the method used for data encoding.

2.2 Construction of the optical circuit

The unitary matrix obtained by the unitary dilation construction is twice as big as the original data. In this context, this would necessitate a 56 modes circuit. In theory, this is not a problem as GPUs provided by Scaleway can compute up to hundreds of modes, but, as explained further, the training step of my model necessitate hundreds of thousands evaluations of the optical circuit, and a circuit with too much modes leads to very lengthy training durations (up to several days).

I have therefore decided to reduce the size of the training data, from 28x28 to 14x14 using max pooling. This will be taken into account in the results section, where the CNN will also be evaluated on 14x14 images for a fair comparison.

The unitary dilation matrix holds all the information about the data in one of its half. Therefore, optical components will only be placed on half of the circuit at the time.

The circuit is constructed as follows:

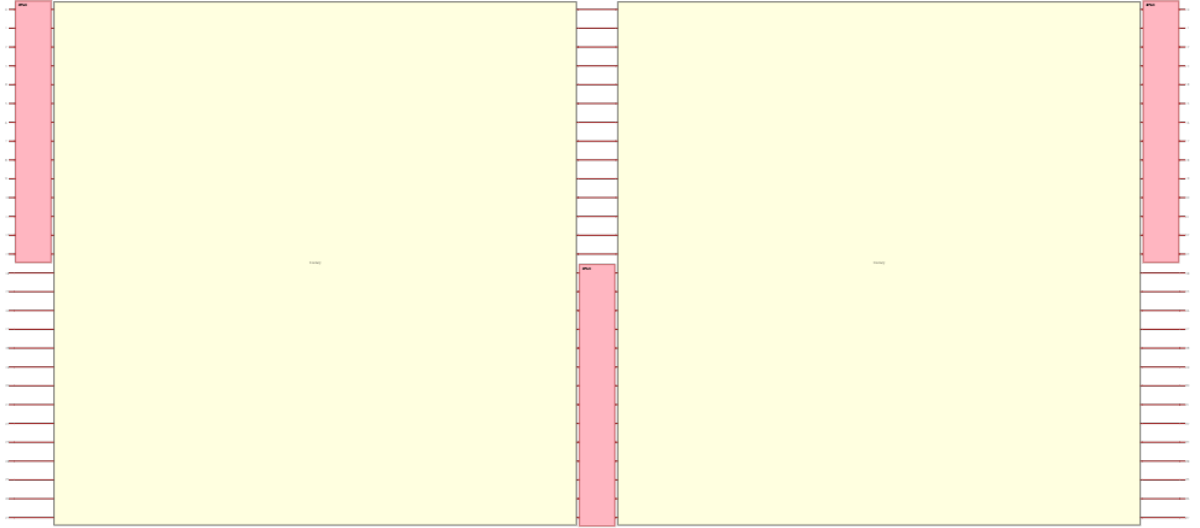


Figure 2: Example circuit with $L=2$. The first red block is a brickwork circuit made with beam splitters. The other red blocks are brickwork circuits made with generic 2-modes circuits.

The input state has a photon in every even modes of the upper half of the circuit. A row of beam splitters is first placed in order to have an optimal spatial input.

Then, there are L layers of a unitary matrix followed by a brickwork construction of generic 2-modes circuits. The brickwork layer is here to reproduce the same effect as a classical convolutional layer, that is extracting features from neighboring input values. A property of the unitary matrix construction that I have found is that the associated circuit tends to shift the photons from one half of the circuit to another, and in order to accommodate with that the brickwork blocs have been placed accordingly. This construction is inspired by [2].

Finally, a post-selection condition so that every photons end up in the same half of the circuit is applied.

2.3 Transforming the optical output into a predicted label

The output of the optical circuit is an array with the number of photons detected in each selected mode. My first idea was to compute an average of the spatial modes weighted by the average number of photons

ending up in the said mode. That way, the model outputs a label closer to 0 if there are a lot of photons closer to the top mode, and outputs a label closer to 9 if there are a lot of photons closer to the bottom mode. This turned out not to be a useful idea, as whatever the probability distribution of the photons is, the central limit theorem forces the outputted label to follow a gaussian distribution centered around the middle values (so around 4 or 5). It is therefore very hard for the model to outputs extreme values like 0 or 9.

A solution to this problem is to not consider a fully-optical model, but rather a hybrid model: an optical part is used to extract useful features from the input data, and a classical part is used to interpret the output of the optical part and turn it into a predicted label.

2.4 Optimization algorithm

2.4.1 Idea for the first output method

I will rapidly go over the ideas I had while working with the spatial weighted average method. As the weighted average was a real value between 0 and 10, I used the MSE loss in order to train the model, and then I would optimize the thresholds in order to turn this real value into a predicted label, so that the training accuracy would be maximized. I found out that the naive approach of the gradient-descent algorithm for the optimization of the optical parameters was too simple, as there was noise in the computed loss due to the fact that I was sampling from the photons spatial distribution. This noise meant that a numerical computation of the gradient was impossible without using a big step.

2.4.2 Idea for the hybrid model

With a classical layer, it is easier to compute a softmax array of prediction probabilities. Therefore I used the categorical cross-entropy loss. As I used TensorFlow for the classical part, i could not update all the parameters at the same time as I needed to use TensorFlow methods for the classical part and algorithms coded from scratch for the optical part. Therefore, instead of considering the model as a whole, I considered it as two subsystems with different learning rates (an analogy is to be made with automatics, where the two subsystems making up the whole system can be considered independent if the eigenvalues of their dynamical equations are different by orders of magnitude, i.e. there is a slow one and a fast one. The fast subsystem can be controlled by a fast controller and the slow one with a slow controller independently without compromising the stability of the whole system).

In order to do this, for each batch of training data, I perform a full optimization step of the classical parameters, and I only update slightly the optical parameters in order to ensure the convergence of the whole model.

For the optimization of the optical parameters, I tried using two algorithms that are robust to noise and require a limited amount of loss function evaluations (for computational efficiency). The algorithms are SPSA (stochastic sub-gradient method) and CMA-ES (optimized genetic algorithm). Because the SPSA algorithm is the fastest, it is the one I mainly focused on.

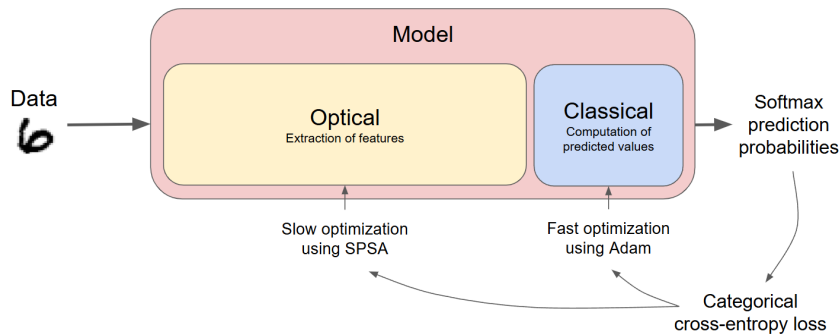


Figure 3: Summary of the training method

3 Results

The criteria chosen for the comparison of the two models are as follows:

- Both models should have a similar amount of parameters (around 500 in this case).
- The input data is 14x14 for both models.
- The performance metrics are the accuracy and the confusion matrix on the validation set.

It is important to note that the training of the classical model is orders of magnitude faster than the training of the optical model. (1 s/epoch vs 1.4 hours/epoch). An important variable in my model is the number of sample used to compute the average photon spatial distribution (which is given to the classical part). I have settled with 5000 samples, as I found it to be the best precision/time ratio. The number of images per batch is 64 for both models.

3.1 Classical CNN model

The CNN I used is constructed using 2 convolutional layers and 3 dense layers. A dropout layer is added to prevent over-fitting. The training is done using the Adam optimizer on 50 epochs.

Layer (type)	Output Shape	Param #
conv2d_40 (Conv2D)	(None, 12, 12, 2)	20
max_pooling2d_40 (MaxPooling2D)	(None, 6, 6, 2)	0
conv2d_41 (Conv2D)	(None, 4, 4, 2)	38
max_pooling2d_41 (MaxPooling2D)	(None, 2, 2, 2)	0
flatten_19 (Flatten)	(None, 8)	0
dropout_2 (Dropout)	(None, 8)	0
dense_58 (Dense)	(None, 6)	54
dense_59 (Dense)	(None, 24)	168
dense_60 (Dense)	(None, 10)	250

Total params: 530 (2.07 KB)

Trainable params: 530 (2.07 KB)

Figure 4: Description of the CNN model considered

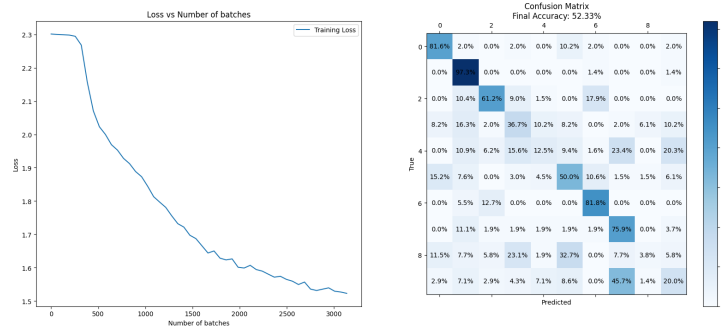


Figure 5: Results of the training of the CNN model

3.2 My hybrid LOQC model

I used $L=6$ for the optical part (325 parameters) and one hidden layer for the classical part (240 parameters) for a total of 565 parameters.

3.2.1 SPSA algorithm

The model has been trained on 5 epochs, though it is not fair to compare the number of epochs between the two models, as the classical part of the model is way more trained due to the separate approach of the optimization algorithm.

```

Training with SPSA          Total number of parameters (optical/classical): 565 (325, 240)
Epoch : 0
Training with SPSA: 100%|██████████| 90/90 [1:18:49<00:00, 52.55s/it]
Total loss: tf.Tensor([1.5406761], shape=(1,), dtype=float32)
Validation accuracy: 0.46735395189003437
Epoch : 1
Training with SPSA: 100%|██████████| 90/90 [1:16:52<00:00, 51.25s/it]
Total loss: tf.Tensor([1.6859304], shape=(1,), dtype=float32)
Validation accuracy: 0.3951890034364261
Epoch : 2
Training with SPSA: 100%|██████████| 90/90 [1:18:00<00:00, 52.00s/it]
Total loss: tf.Tensor([1.7105218], shape=(1,), dtype=float32)
Validation accuracy: 0.436426116838488
Epoch : 3
Training with SPSA: 100%|██████████| 90/90 [1:19:19<00:00, 52.89s/it]
Total loss: tf.Tensor([1.5233896], shape=(1,), dtype=float32)
Validation accuracy: 0.42955326460481097
Epoch : 4
Training with SPSA: 100%|██████████| 90/90 [1:17:35<00:00, 51.73s/it]
Total loss: tf.Tensor([1.515729], shape=(1,), dtype=float32)
Validation accuracy: 0.41408934707903783

```

Figure 6: Results of the hybrid model training using SPSA

I have not saved the parameters after this training so I have not been able to plot the confusion matrix.

3.2.2 CMA-ES algorithm

The CMA-ES algorithm is a genetic algorithm, so it requires more loss function evaluations than the SPSA algorithm but every minimization steps outputs a lower value for the loss function, which is not the case for the SPSA algorithm. As one can see, the evolution of the loss using SPSA is not monotonous, which can be expected for a stochastic method. Therefore I believe that CMA-ES is a better option for few parameters updates, as it is the case right now, but on my current hardware it takes about 10 hours per epochs. Sadly, I haven't been able to use the full potential of CMA-ES.

3.2.3 Attempt with only 500 samples per evaluation

With the hope that the lower precision given by fewer samples can be compensated by the classical part, I have tried to run the previous algorithms for more epochs.

Here are the results for 3 epochs with SPSA:

```

Training with SPSA          Total number of parameters (optical/classical): 565 (325, 240)
Epoch : 0
Training with SPSA: 100%|██████████| 90/90 [40:18<00:00, 26.87s/it]
Total loss: 1.6817352679673911
Validation accuracy: 0.3833049403747871
Epoch : 1
Training with SPSA: 100%|██████████| 90/90 [37:22<00:00, 24.92s/it]
Total loss: 1.7315113525482182
Validation accuracy: 0.36626916524701875
Epoch : 2
Training with SPSA: 100%|██████████| 90/90 [37:13<00:00, 24.81s/it]
Total loss: 1.6120425854856228
Validation accuracy: 0.40545144804088584

```

Figure 7: Results of the hybrid model training using SPSA with 500 samples

The loss is in general slightly higher and the validation accuracy slightly lower with 500 vs 5000 samples per evaluation.

The evolution of the lost function evaluated on the batch it is being trained on is, for the most part, monotonous, which is a sign that the minimization algorithm seems to be working. However, there are big spikes when a new epoch is started that throw off the minimization algorithm. I haven't yet found an explanation for that.

The confusion matrix behaves similarly between the CNN and the hybrid model. 1 is the easiest category, while 8 is very hard.

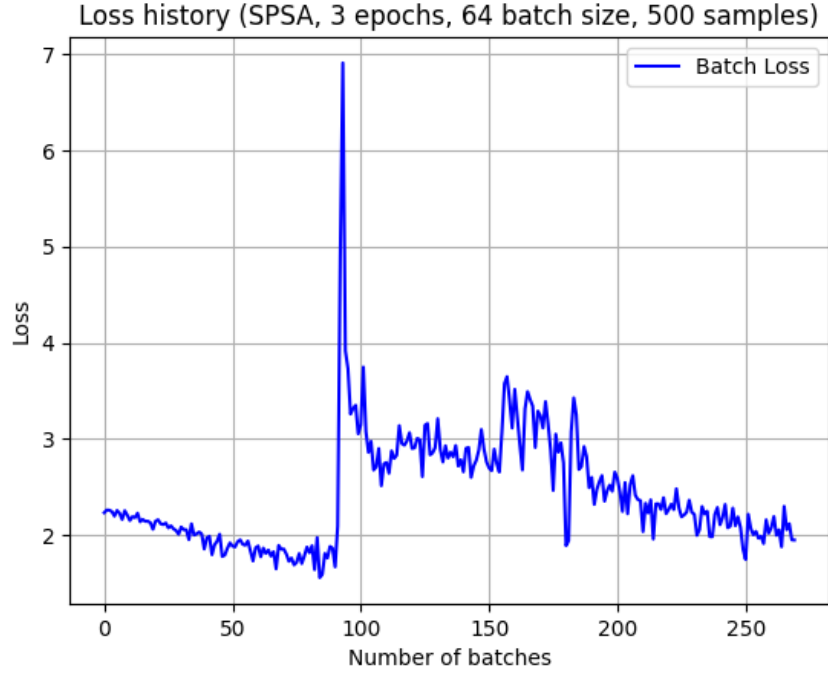


Figure 8: Enter Caption

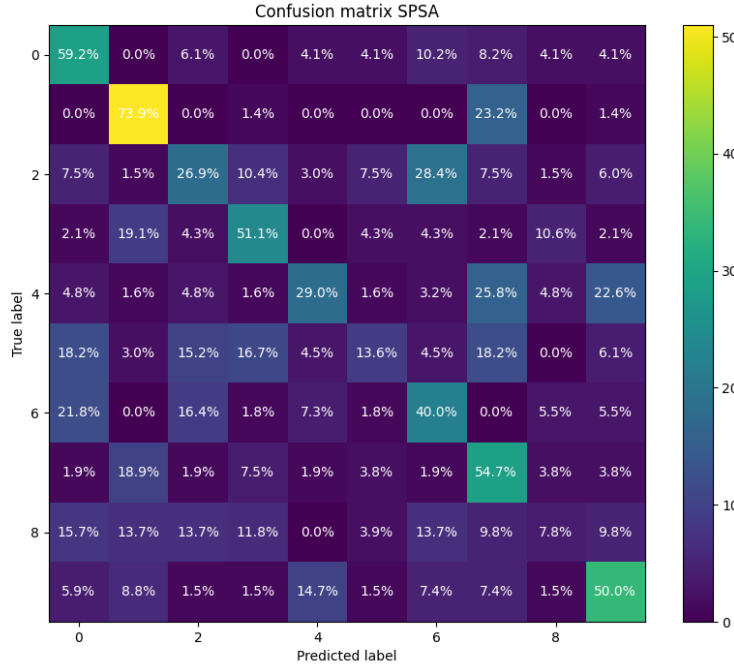


Figure 9: Confusion matrix of trained model with SPSA, 3 epochs, 500 samples

4 Conclusion

Both models are reaching similar loss function values (around 1.5), though the validation accuracy is better with the CNN model. An explanation could be that the hybrid model is over-fitting.

It is important to note that the optical part of the hybrid model has surely not been used to its full potential, as it has undergone only a few updates of its parameters : the SPSA algorithm is stochastic,

and converges only if several steps are done. Maybe a better approach would have been to stick to a classical gradient descent algorithm with a fast implementation using already existing python library. I believe that the optical parameters optimization is the weak point of my work, and going for algorithm that only uses few function evaluations might not be the best idea, but it was necessary for computational reasons.

However, I hope that the work presented here can be helpful, as there are some hints that hybrid models can outperform classical models in accuracy with a lower number of parameters.

I would like to thank Quandela and Scaleway for the opportunity to learn about quantum computing and its implementation. A special thanks Jean Senellart for his invaluable help in deciphering the mysteries that arose while running my Python file.

References

- [1] Mezher, R., Carvalho, A. F., & Mansfield, S. (2023). Solving graph problems with single-photons and linear optics (No. arXiv:2301.09594). arXiv. <https://doi.org/10.48550/arXiv.2301.09594>
- [2] Wang, P., Myers, C. R., Hollenberg, L. C. L., & Parampalli, U. (2024). Quantum Hamiltonian Embedding of Images for Data Reuploading Classifiers (No. arXiv:2407.14055). arXiv. <https://doi.org/10.48550/arXiv.2407.14055>